Design & Analysis of Algorithms

LECTURE 04

Fundamentals of the Analysis of Algorithm Efficiency

CHAPTER 02

Analysis Framework

- We already discussed that there are two kinds of efficiency:
- **Time efficiency**, also called time complexity, indicates how fast an algorithm in question runs
- **Space efficiency**, also called space complexity, refers to the amount of memory units required by the algorithm in addition to the space needed for its input and output.
- In the early days of electronic computing, both resources—time and space—were at a premium. Half a century of relentless technological innovations have improved the computer's speed and memory size by many orders of magnitude. Now the amount of extra space required by an algorithm is typically not of as much concern.
- The time issue has not diminished quite to the same extent, however. In addition, the research experience has shown that for most problems, we can achieve much more spectacular progress in speed than in space. Therefore, our primary concentration will be Time Efficiency.

Measuring an Input Size

It is obvious observation that almost all algorithms run longer on larger inputs. For example, it takes longer to sort larger arrays, multiply larger matrices, and so on.

Therefore, it is logical to investigate an algorithm's efficiency as a function of some parameter n indicating the algorithm's input size.

There are situations, of course, where the choice of a parameter indicating an input size does matter. One such example is computing the product of two n × n matrices.

Units for Measuring Running Time

- The next issue concerns units for measuring an algorithm's running time.
- Of course, we can simply use some standard unit of time measurement—a second, or millisecond, and so on—to measure the running time of a program implementing the algorithm.
- There are obvious drawbacks to such an approach, however: dependence on the speed of a particular computer, dependence on the quality of a program implementing the algorithm and of the compiler used in generating the machine code, and the difficulty of clocking the actual running time of the program.
- Since we are after a measure of an algorithm's efficiency, we would like to have a metric that does not depend on these extraneous factors.

Units for Measuring Running Time

- Count No. of Times- But Difficult/Unnecessary
- Identify most important Operation – Contributing more
- Count only Basic operation
- e.g For Sorting Algorithm -- Comparison is basic operation
- So, an algorithm's time efficiency suggests measuring it by counting the number of times the algorithm's basic operation is executed on inputs of size n.
- $T(n) \approx c^{op} C(n)$

Orders of Growth

The order of growth of an algorithm is an approximation of the time required to run a computer program as the input size increases.

The order of growth ignores the constant factor needed for fixed operations and focuses instead on the operations that increase proportional to input size.

What is a Time Complexity/Order of Growth?

Time Complexity/Order of Growth **defines the amount of time taken by any program with respect to the size of the input**. Time Complexity specifies how the program would behave as the order of size of input is increased

Orders of Growth

Suppose you have analyzed two algorithms and expressed their run times in terms of the size of the input: Algorithm A takes $\mathbf{100}n\mathbf{+1}$ steps to solve a problem with size n; Algorithm B takes $n^2 + n + 1$ steps.

The following table shows the run time of these algorithms for different problem sizes:

Input Size	**Run Time of Algorithm A**	**Run Time of Algorithm B**
10	1,001	111
100	10,001	10,101
1,000	100,001	1,001,001
10,000	1,000,001	$> 10^{10}$

Important Functions

These functions often appear in algorithm analysis:

Function	Name
c	Constant
$\log N$	Logarithmic
$\log^2 N$	Log-squared
N	Linear
$N \log N$	N log N
N^2	Quadratic
N^3	Cubic
2^N	Exponential
$N!$	
N^N	

Functions in order of increasing growth rate

Performance Classification

f(*n*)	Classification
1	*Constant*: **run time is fixed, and does not depend upon n**. Most instructions are executed once, or only a few times, regardless of the amount of information being processed
log n	*Logarithmic*: **when *n* increases, so does run time, but much slower**. Common in programs which solve large problems by transforming them into smaller problems.
n	*Linear*: **run time varies directly with *n***. *Typically, a small amount of processing is done on each element.*
n log n	**When *n* doubles, run time slightly more than doubles**. *Common in programs which break a problem down into smaller sub-problems, solves them independently, then combines solutions*
n^2	***Quadratic*: when *n* doubles, runtime increases fourfold.** Practical only for small problems; typically the program processes all pairs of input (e.g. in a double nested loop).
n^3	***Cubic*: when n doubles, runtime increases eightfold**
2^n	*Exponential*: **when n doubles, run time squares**. This is often the result of a natural, "brute force" solution.

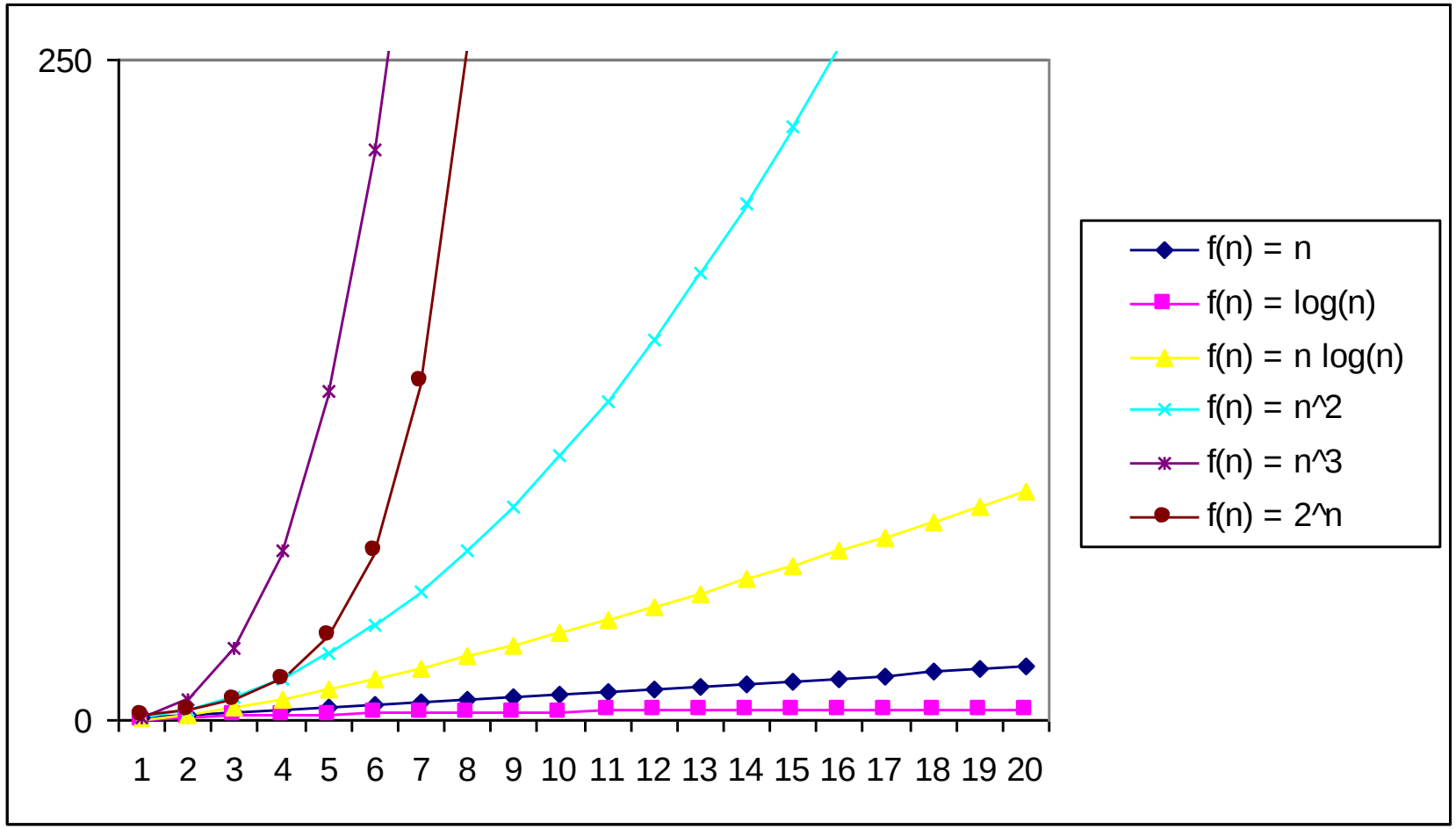
A comparison of Growth-Rate Functions

Size does Matter:

What happens if we increase the input size N?

Function	n = 10	100	1,000	10,000	100,000	1,000,000
1	1	1	1	1	1	1
$\log_2 n$	3	6	9	13	16	19
n	10	10^2	10^3	10^4	10^5	10^6
$n * \log_2 n$	30	664	9,965	10^5	10^6	10^7
n^2	10^2	10^4	10^6	10^8	10^{10}	10^{12}
n^3	10^3	10^6	10^9	10^{12}	10^{15}	10^{18}
2^n	10^3	10^{30}	10^{301}	$10^{3,010}$	$10^{30,103}$	$10^{301,030}$

A comparison of Growth-Rate Functions



Worst case, Best case and Average case Efficiencies

In the beginning of this section, we established that it is reasonable to measure an algorithm's efficiency as a function of a parameter indicating the size of the algorithm's input.

But there are many algorithms for which running time depends not only on an input size but also on the specifics of a particular input.

Consider, as an example, sequential search. This is a straightforward algorithm that searches for a given item (some search key K) in a list of n elements by checking successive elements of the list until either a match with the search key is found or the list is exhausted.

Worst case, Best case and Average case Efficiencies

ALGORITHM *SequentialSearch*$(A[0..n-1], K)$

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n − 1] and a search key K
//Output: The index of the first element in A that matches K
//              or −1 if there are no matching elements
i ← 0
while i < n and A[i] ≠ K do
      i ← i + 1
if i < n return i
else return −1
```

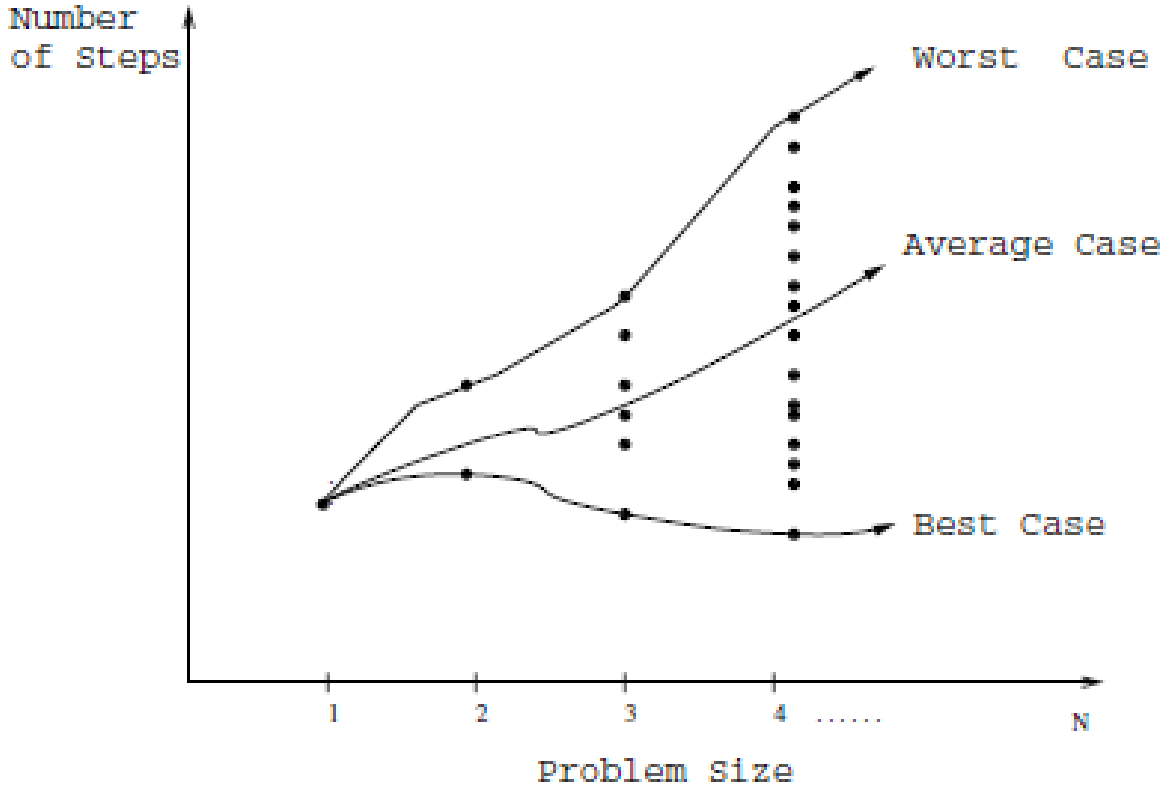
Worst case, Best case and Average case Efficiencies

Best Case : When an Element found in an Array at the starting point

Worst Case : Element found at last

Average Case: Element found in somewhere at the middle

Worst case, Best case and Average case Efficiencies



Asymptotic Notations

NEXT LECTURE

Questions

THANK YOU.